# TermiCode AI — Technical Portfolio & System Architecture

**Author / Candidate:** Bhavesh Bargat | **Target Roles:** AI Engineer · AI/ML Engineer · Software Engineer

**GitHub:** github.com/Bhaveshbargat1/Termicode_AI | **Package:** termicode v0.1.0 | **Verification:** 288 Automated Tests (100% Pass)

## 1. Project Overview & Problem Statement

**TermiCode AI** is an open-source, bounded terminal coding assistant designed around the core principle of **deterministic authority separation**. It bridges generative Large Language Models (LLMs) with local software development workflows while eliminating the severe security risks associated with autonomous code execution.

**The Problem:** Traditional autonomous agents grant untrusted LLMs direct shell access (shell=True) and unbounded filesystem privileges. This exposes developers to accidental catastrophic deletion (rm -rf /), directory traversal attacks accessing sensitive credentials (~/.ssh/id_rsa, .env), infinite token consumption loops, and silent code overwrites without human verification.

**The Solution:** TermiCode AI strictly treats the LLM as an untrusted proposal engine. Python deterministic software boundaries validate safety policies, confine all file operations to an approved workspace boundary, enforce human-in-the-loop visual unified diff approvals, and execute allowlisted subprocesses with shell=False.

## 2. Complete Technology Stack & Rationale

| Category | Technology | Version / Source | Architectural Rationale |
|---|---|---|---|
| Core Language | Python | >=3.11 | Modern typed unions, match syntax, asyncio, and StrEnum. |
| CLI Framework | Typer | >=0.12.0 | Type-annotated CLI parsing, flags, autocompletion, and subcommands. |
| Terminal UI | Rich | >=13.0.0 | Status panels, execution tables, unified diff syntax highlighting. |
| Validation | Pydantic | >=2.7.0 | Runtime data validation, tool schemas, and JSON serialization. |
| Settings | Pydantic-Settings | >=2.2.0 | Automated loading of TERMICODE_* env vars and validation bounds. |
| HTTP Client | urllib.request | Standard Library | Zero external dependencies; raw HTTP to OpenAI and Ollama. |
| Safety Bounds | pathlib & subprocess | Standard Library | Canonical path resolution (resolve()) and process isolation (shell=False). |
| Testing Suite | Pytest | >=8.0.0 | 288 automated regression tests covering agents, policy, and tools. |
| Quality & Type | Ruff & Mypy | Strict Mode | 100% type coverage across 38 source files; clean 100-char style. |

## 3. System Architecture & Authority Separation

TermiCode AI decouples cognitive decision-making from execution authority across 5 distinct operational layers:

### Core Architectural Layers

- **Layer 1 — CLI & Terminal UI (cli.py, console.py):** Ingests user tasks, manages subcommands (run, understand, generate, debug, tests, refactor), and renders Rich formatted tables.
- **Layer 2 — Safety Fence (workspace.py, config.py):** Canonical path resolution anchors all actions strictly to the project root and protects sensitive credentials (.env, .git, SSH keys).
- **Layer 3 — Bounded Agent Loop (runtime.py, workflows.py):** Event-driven state machine managing step budgets (8–12 steps), repair attempts (max 3), and context history.
- **Layer 4 — Policy & Approval Gate (policy.py, cli.py):** Categorizes actions into READ, WRITE, EXECUTE. Displays unified diff previews and strictly defaults to DENY on invalid input.
- **Layer 5 — Tool Registry & Sandboxed Runner (tools/, runner/):** Executes allowlisted binaries (python, pytest, ruff, mypy, git) with shell=False, 30s timeout, and 64KB buffer caps.

## 4. End-to-End Execution Workflow

When a user runs termicode generate "create src/utils/math_helper.py", the application executes the following deterministic sequence:

### Execution Lifecycle Trace

1. **Ingestion:** cli.py receives the task and options, initializing WorkspaceBoundary(Path.cwd()).
2. **Profile Loading:** workflows.py loads the generate profile (12 steps, risk ceiling: WRITE, system prompt).
3. **Provider Selection:** Probes local Ollama endpoint (http://localhost:11434/v1) or validates OPENAI_API_KEY with interactive offline fallback.
4. **AI Proposal:** AgentRuntime dispatches context and Pydantic tool schemas. Model returns ToolCall(name="write_file", arguments={...}).
5. **Policy Evaluation:** PolicyEngine verifies the path is inside workspace and marks status as needs_approval.
6. **Human Gate:** cli.py renders a visual unified diff (red deletions, green additions) and prompts: Approve action? [y/N].
7. **Tool Execution:** Upon typing 'y', WriteFileTool creates the parent directory and writes the file to disk.
8. **Context & Audit:** TranscriptWriter logs the event to .termicode/transcripts/run_*.jsonl with secrets scrubbed. Tool output is appended to message history.
9. **Completion:** Model sees the successful result, completes without tool calls, and cli.py prints the Rich Execution Summary.

## 5. Feature Catalog & Workflow Profiles

| Workflow Profile | Allowed Risk | Step Budget | Primary Objective & Permission Matrix |
|---|---|---|---|
| understand | READ only | 8 steps | Codebase comprehension, AST exploration, file structure analysis. Mutating tools strictly excluded. |
| review | READ only | 8 steps | Static code review, security audits, linting inspection. Zero disk writes allowed. |
| generate | READ, WRITE | 12 steps | Feature implementation, module creation, and test authoring with unified diff preview. |
| debug | READ, WRITE, EXECUTE | 10 steps | Bug diagnosis, targeted patch application, and verification via allowlisted test runner. |
| tests | READ, WRITE, EXECUTE | 10 steps | Test suite discovery, test generation, and running pytest under subprocess supervision. |
| refactor | READ, WRITE, EXECUTE | 10 steps | Structural code modernization, deduplication, and dependency decoupling with regression checks. |

## 6. AI Integration & Local GPU Deployment

TermiCode AI implements a hybrid inference topology supporting both cloud APIs and 100% private local GPU models:

### AI Integration & Inference Strategy

- **Cloud Inference:** Connects to OpenAI (gpt-4o-mini, gpt-4o) via HTTP POST using Python's standard urllib.request.
- **Local GPU Inference (Ollama):** Auto-detects local Ollama instances on http://localhost:11434/v1. Highly optimized for consumer GPUs like the **NVIDIA GeForce RTX 3050 6GB** using qwen2.5-coder:7b (~4.7 GB VRAM at 40-60 tokens/sec).
- **Resilient Fallback Parser:** Smaller open-weights models occasionally emit tool calls formatted in markdown (```json or <tool_call>). OpenAIProvider._extract_fallback_tool_call() uses regex to extract and parse them into valid ToolCall objects.
- **Smart Offline Engine:** Passing --offline routes execution to a context-aware simulation engine that inspects real workspace files and generates typed, production-ready Python modules with zero external calls.

## 7. Security Architecture & Threat Mitigation Matrix

| Security Threat | Mitigation Mechanism | Implementation File | Defense Reality & Guarantees |
|---|---|---|---|
| Directory Traversal (../../) | Canonical Path Resolution | workspace.py: resolve_safe() | Resolves absolute symlinks via Path.resolve() and checks is_relative_to(). Escapes blocked with WorkspaceEscapeError. |
| Credential Theft (.env, SSH) | Sensitive File Blocklist | workspace.py: is_sensitive() | Blocks access to .git, .ssh, .aws, .env, id_rsa, and *.key files even inside the approved workspace root. |
| Shell Injection Attacks | Subprocess Disabling Shell | runner.py: CommandRunner | Runs commands with shell=False. Metacharacters (\|, ;, &&, $()) are treated as literal strings and blocked by validator. |
| Arbitrary Process Spawning | Strict Binary Allowlist | runner/validator.py | Only 5 binaries permitted: python, pytest, ruff, mypy, git. Destructive git subcommands (push, reset, rm) blocked. |
| Accidental File Overwrite | Interactive Diff Approval | cli.py & policy.py | Displays colored unified diffs before writes. Defaults strictly to DENY on invalid input or non-interactive sessions. |
| Credential Leakage in Logs | Automated Secret Redaction | providers/redaction.py | Deterministic regex scrubber masks sk-*, AIza*, and bearer tokens before writing to transcripts or summaries. |

**Honest MVP Limitations:** TermiCode AI enforces application-level policy boundaries and human approval gates. It does *not* provide OS kernel-level sandboxing (no Docker/seccomp containers). Permitted tools run under the user's host account.

## 8. Testing, Quality & Performance Verification

### Verified Code Quality Metrics

- **Automated Pytest Suite: 288 collected test items** across 12 test modules. **287 passed, 1 skipped** (live Ollama test requiring daemon) in **9.81s**.
- **Static Type Analysis (Mypy):** Enforced under strict = true, disallow_untyped_defs = true. Result: **Success: no issues found in 38 source files**.
- **Linter & Formatter (Ruff):** Enforced under 100-character line lengths, pyflakes, pycodestyle, isort, and bugbear rules. Result: **All checks passed cleanly**.
- **Regression Coverage:** Comprehensive tests for agent loops, step budgets, repair counters, tool validation, workspace path escapes, and secret redaction.

## 9. Professional Resume Descriptions & Technical Keywords

### Resume & Portfolio Presentation

**Professional Project Description:**
*TermiCode AI is an open-source, bounded terminal coding assistant built with Python and Typer that enforces deterministic authority separation between Large Language Model planners and local system execution. It enables safe local code generation, debugging, and testing through canonical workspace confinement, unified diff approval gates, and allowlisted subprocess sandboxing supporting cloud and local (Ollama) inference.*

### High-Impact Technical Resume Bullet Points:

- **Architected a secure, bounded AI agent runtime** in Python 3.11 with an event-driven propose-validate-approve-execute state machine, enforcing strict step budgets and automatic repair cycles across 6 distinct workflow profiles.
- **Engineered deterministic security guardrails** utilizing pathlib canonicalization to prevent path traversal attacks and subprocess execution with shell=False to eliminate shell injection vulnerabilities.
- **Built a zero-dependency OpenAI-compatible provider adapter** using standard library urllib.request, enabling seamless switching between OpenAI cloud APIs, local GPU-accelerated Ollama models, and an offline testbed.
- **Implemented interactive human-in-the-loop approval gates** with rich and difflib, rendering visual unified diff previews with fail-safe default-to-deny authorization and automated secret redaction.
- **Achieved 100% test pass rate across 288 automated pytest suites** and zero static analysis issues under strict mypy typing and ruff linting across 38 source files.

**ATS-Friendly Keywords:** Python 3.11, Agentic Workflows, Tool Calling, Function Calling, Ollama, OpenAI API, Qwen2.5-Coder, Deterministic Policy, Path Traversal Defense, Shell Injection Defense, Human-in-the-Loop, Typer, Rich, Pydantic v2, Pytest, Mypy, Ruff, Subprocess Security, Secret Redaction.

## 10. Technical Interview Preparation (15 Core Q&A;)

**Q1: What is 'authority separation' in TermiCode AI?**

Authority separation decouples the thinking entity from the executing entity. The LLM acts as an untrusted planner that can only propose actions in structured JSON. Deterministic Python policy code evaluates whether the action is safe and asks the developer for confirmation before any tool modifies files or executes commands.

**Q2: Why did you implement shell=False in subprocess execution?**

When shell=True, the OS invokes a shell interpreter (sh/cmd.exe) that evaluates characters like ;, |, &&, and $(), enabling command injection. Using shell=False and passing arguments as an explicit array (['pytest', '-v']) ensures the OS treats every token literally, eliminating injection vulnerabilities.

**Q3: How do you prevent an AI model from reading files outside the project directory?**

Through WorkspaceBoundary.resolve_safe(). It resolves target paths to canonical absolute paths via pathlib.Path.resolve(), resolving symlinks and ../ traversals, then checks resolved.is_relative_to(workspace_root). If false, it immediately raises a WorkspaceEscapeError.

**Q4: Why build an OpenAI-compatible adapter with urllib.request instead of the official SDK?**

To eliminate heavy external dependency graphs and potential version conflicts. Standard library urllib.request allows TermiCode to speak the standard OpenAI REST protocol (/chat/completions) directly to both OpenAI cloud servers and local Ollama servers with zero third-party overhead.

**Q5: What happens if an AI agent gets stuck in an infinite repair loop?**

AgentRuntime enforces two deterministic bounds: max_steps (8–12 steps depending on workflow profile) and max_repairs (maximum of 3 consecutive repair attempts). On the 4th consecutive failure, execution terminates immediately with StopReason.MAX_REPAIRS_REACHED.

**Q6: What is the purpose of the 6 workflow profiles?**

To enforce the principle of least privilege. In read-only workflows like understand and review, mutating tools (write_file, apply_patch, run_command) are completely excluded from tool declarations sent to the model, and the PolicyEngine strictly denies any mutating attempts.

**Q7: How does your tool registry connect to the AI model?**

Tools inherit from BaseTool and define a Pydantic args_schema. ToolRegistry calls args_schema.model_json_schema() to generate OpenAI-compatible JSON function declarations that inform the model of tool names, descriptions, and required argument types.

**Q8: How does the interactive approval gate protect against accidental file overwrites?**

For mutating tools (write_file, apply_patch), the CLI computes a unified diff between current disk content and proposed content using difflib. It displays the colored diff in a Rich panel and prompts the user. If the user enters anything other than 'y' or 'yes', it defaults strictly to DENY.

**Q9: What happens if TermiCode runs in a non-interactive CI/CD pipeline?**

cli.py checks sys.stdin.isatty(). In non-interactive or piped sessions, approval prompts automatically default to DENY unless explicitly run with the --yes flag, preventing automated pipelines from hanging indefinitely.

**Q10: How do you protect sensitive credentials like .env files from being exposed to the AI?**

WorkspaceBoundary.is_sensitive() checks file paths against a blocklist of sensitive directories (.git, .ssh, .aws), filenames (.env, credentials.json, id_rsa), and extensions (.pem, .key), raising SensitivePathError. Additionally, redact_secrets() masks API keys before writing transcripts.

**Q11: How does the application handle local models that output tool calls in markdown rather than JSON?**

OpenAIProvider._extract_fallback_tool_call() uses regular expressions to capture or ```json blocks containing 'name' and 'arguments', parsing them into valid ToolCall objects to maintain compatibility with smaller open-weights models.

**Q12: Why did you choose flat-file JSONL transcripts over an SQLite database?**

For a local developer CLI, transcripts must be human-readable, zero-configuration, and easily inspectable in any text editor. JSON Lines allows append-only logging in under a millisecond without database locks and is easily wiped per repository without schema migrations.

**Q13: What are the command execution restrictions in validate_command?**

Only 5 executables are allowlisted: python, pytest, ruff, mypy, git. Git is restricted strictly to read-only subcommands (status, diff, log, branch, show), and Python execution is limited to approved test and lint modules (pytest, unittest, ruff, mypy).

**Q14: What is the purpose of the --offline flag?**

It routes execution to _create_smart_offline_provider(), which dynamically inspects workspace files and produces typed, production-ready Python modules and architecture summaries without requiring an internet connection or an active LLM server.

**Q15: What are the honest security limitations of TermiCode AI?**

TermiCode provides application-level policy boundaries, not kernel-level virtualization (no Docker/seccomp containers). Approved commands execute under the user's host permissions, and safety relies on developer verification before approving diffs.